

UNIT 13 SMART CLIENT AND ENTERPRISE SERVER BASED ARCHITECTURE

Structure

- 13.0 Introduction
- 13.1 Objectives
- 13.2 Introduction to Smart Client Architecture
 - 13.2.1 Smart Client Architecture
 - 13.2.2 Differences between the Architectures
 - 13.2.3 User Interface of Smart Client
- 13.3 Data Synchronization Formats
 - 13.3.1 Open Standard Formats
 - 13.3.2 Flat File format
 - 13.3.3 Database Format
 - 13.3.4 Device specific Storage
- 13.4 Data synchronization at Clients and Servers
 - 13.4.1 Synchronization Types
 - 13.4.2 Synchronization Usage Models
- 13.5 Mobile Devices Support Infrastructure and Management
 - 13.5.1 Mobile Device Support Infrastructure
- 13.6 Summary
- 13.7 Solutions/Answers
- 13.8 Further Readings

13.0 INTRODUCTION

13.1 OBJECTIVES

After going through this unit, you should be able to

- understand key concepts of mobile data management,
- understand data replication schemes,
- understand key concepts of adaptive clustering
- Understand the mobile device support infrastructure.

13.2 INTRODUCTION TO SMART CLIENT ARCHITECTURE

In a typical n-tier application a web application client collects the data from the end user. For instance, an end user fills up the registration form and submits the form. The web client then sends the data in the prescribed format (such as JSON) to the server.

At the server end, we have the server components such as load balancer and web servers. The web server runs a microservice that validates the data and stores the data in the database. During the registration process the system also generates a unique customer registration id.

The Smart client sends the request to the server which aggregates the response from various data stores. Optionally we use gateways at the sender end and at the

receiver end which is responsible for data transmission, protocol conversion and data transformation.

13.2.1 Smart Client Architecture

In a typical 3-tier enterprise application, we have separate software components/modules to manage the presentation (such as React, Angular, JSP, PHP, .Net), business logic (such as .Net, Spring Boot, NodeJS and others) and database interactions. Let us look at how thick client, thin client and smart client applications manage the modules.

There are multiple ways to develop client software for mobile platforms. In this section we discuss the three main approaches.

Thick Client Architecture

The thick client architecture encompasses all the tier components in a single structure. We have depicted the thick client in Figure 1. Given below are the main tenets of thick client architecture –

- The solution components of all tiers are present as part of single thick client
- Thick client does not communicate online to get the data required for the application to function.
- The thick client components are tightly coupled.

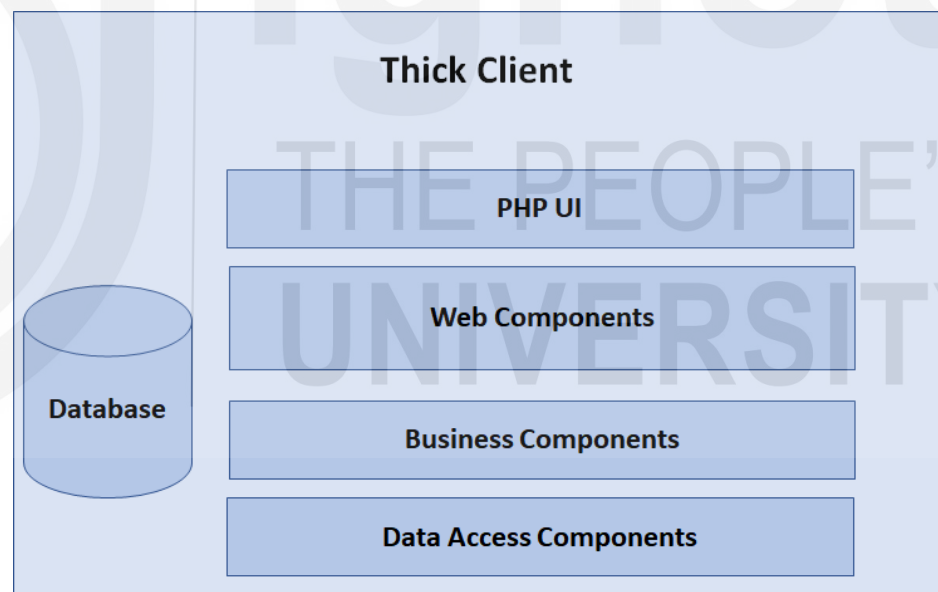


Figure 1 Thick Client Architecture

An example for a thick client is a mobile gaming app that has the components from all the tiers in a single app. The gaming app does not use online services.

Thin Client Architecture

Thin client architecture involves minimal components on the presentation tier as depicted in Figure 2. Given below are the key tenets of thin client architecture –

- The presentation tier has only the components required to render the user interface. For instance, web pages, images, stylesheets, JavaScript libraries.

- Thin client invokes backend services for all the required information.
- The data required for the presentation is retrieved through the backend service call.

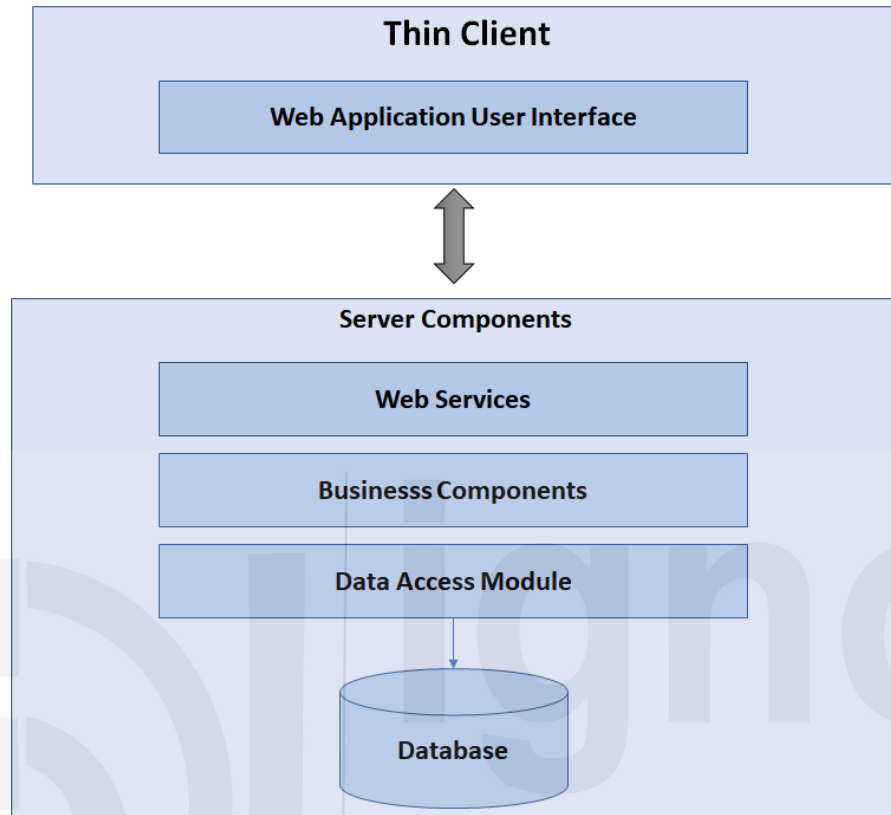


Figure 2 Thin Client Architecture

An example of thin client architecture is the web platform that invokes backend services for all the required data.

Smart Client Architecture

Smart client applications are thin clients that use local database storage for managing the data. Smart clients enable us to implement offline-first approach wherein the applications work in limited network conditions and during no network conditions by using the local storage. Smart clients synchronize the data to the servers when the network is available. Smart client applications are also called rich internet applications as they provide engaging user experience. For instance, a gaming mobile app that ranks the players based on their game score, stores the game score in local database when the network is not available and later synchronizes to the server when the network becomes available.

We have given the smart client architecture in Figure 3. Given below are the key tenets of the smart client architecture –

- The smart client runs on the mobile device and connects to Internet through mobile carrier.
- The data is retrieved from the database server and synchronization server synchronizes the data across database instances.
- The performance of the smart client is optimized as it uses local data cache.

The main challenge in the smart client is the heterogeneity of the mobile platforms. Normally popular mobile smart clients are developed in both android and Apple iOS platforms. Alternatively, we can use hybrid mobile platforms such as Kotlin.

Given below are the main advantages of the smart client architecture -

- It provides rich user interface
- Provides effective user experience
- Leverages the native mobile features such as camera, sensors and such
- Ensure high security
- Can work in low bandwidth conditions
- Provides the rich user experience of thick client and the light weight design of the thin client application.

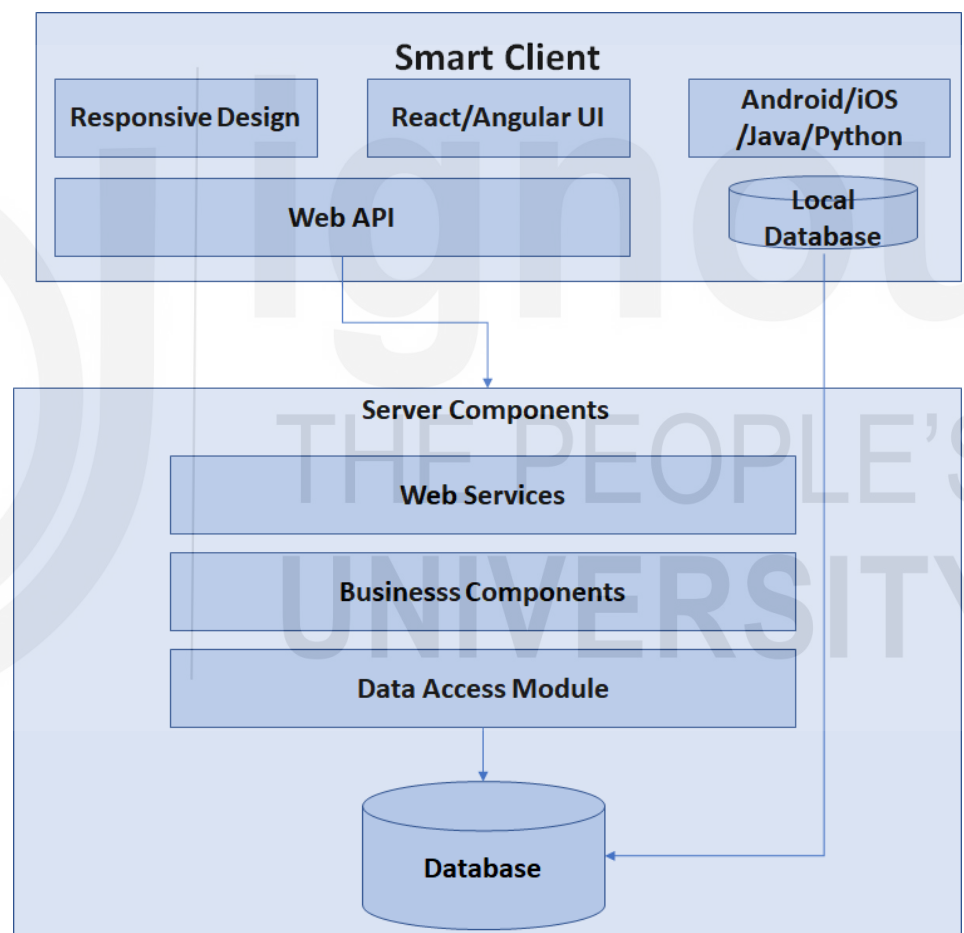


Figure 3 Smart Client Architecture

13.2.2 Differences between the Architectures

Table 1 provides the key differences between thick client, thin client and smart client.

Table 1 Thick Client vs Thin Client vs Smart Client

	Thick Client	Thin Client	Smart client
Data sync	No data sync	No data sync	Frequent data sync
Connectivity	No connectivity	Frequent connectivity	Synchronize local data with remote DB

Advantages	Rich user experience	Light weight design	Rich UI and dynamic data
Disadvantages	Large size without any dynamic data	Frequent server calls & low performance	Not applicable
Size	Large (in MBs)	Small (in KBs)	Small (in KBs)
Example	Windows applications	PHP, JSP, .Net based applications	React/Angular/Vue based applications with local data store

13.2.3 User Interface of Smart Client

The user interface of smart clients should engage the end user. We have given the key design goals for the smart client user interface –

- The user interface should be simple and consistent.
- The user interface should provide intuitive information architecture.
- The user interface should be accessible to all users.
- The user interface should be personalized based on user goals, needs and wants.
- Users should be able to quickly find the relevant information.

The user interface for smart clients use various pattern such as atomic design pattern, client-side UI composition pattern and such. In these patterns we create the larger (or higher order) user elements using smaller and reusable library of user elements. For instance, the mobile screen consists of form and menu UI elements. The form UI element is further composed of text items, drop-down list, radio button, submit button, checkbox and other reusable library of UI elements.

The advantage of using atomic design pattern and client side UI composition pattern are as follows

- We can implement a consistent user experience across the entire application
- We can reuse the library of UI components
- We can build standards based, consistent user experience.

13.3 DATA SYNCHRONIZATION FORMATS

Mobile devices often synchronize the local data with remote servers to prevent the data loss. We have specified the main advantages of data synchronization below –

- Ensure high availability of data across different devices.
- Backup the data in remote device
- Restore the back during data loss

The data format in the source device (such as mobile device) and the target device (such as remote server or PC) might be different. In such case we need to convert the source file format into the target file format.

The data synchronization can be scheduled or automated or can be user based. We can schedule the data synchronization from the mobile device to the remote server based on

the importance of the file. We can also specify the auto-synchronization policy. For instance, when the mobile device connects to the PC through USB, the data is automatically synchronized. The mobile user can also choose to selectively synchronize the data of interest. The user can look at the update report that details the updated/deleted/added data on the source and target device and then chose to synchronize the data. The user can also use conflict resolution policies to update or override the data if the data is updated both on source and the target.

In this section we shall look at common data synchronization formats.

13.3.1 Open Standard Formats

Modern mobile devices support the open standards such as XML, JSON, VCF based data synchronization. Initially we perform the full backup and then we synchronize the incremental changes on daily basis. For instance, the contact list on the mobile device can be synchronized and shared using virtual card format (VCF).

13.3.2 Flat File Format

Flat files such as PDFs, Microsoft word document, Image files (PNG, JPG) are synchronized from the source device to the target device.

While synchronizing the flat files, the entire file is synchronized as many of these file formats are in proprietary binary format. The modified timestamp is used to identify the latest copy of the file and it will be synchronized with the target device.

13.3.3 Database Format

Many mobile devices store the information in its local database that consist of data records and indexed by the key columns. For instance a user profile information table has indexes on user name and phone number or email address as these are the commonly used fields. The indexes are used to speed up the database queries.

When we synchronize the data to the database record we can update individual table rows which are updated. For instance, we can update only the phone number of the user when the user changes the mobile number.

13.3.4 Device Specific Storage

Each of the mobile device has its own format for specific functions. Given below are few examples

- Android platform supports AAC LC, MIDI, MP3 for audio whereas Apple devices support AAC (Apple audio communication).
- Android uses vCalendar file format for calendar. The calendar can optionally contain vtodo, vjournal components.

☛ Check Your Progress 1

1. The architecture that includes all solution components in a single tier is called _____
2. The key difference between thin client and smart client is _____
3. The main disadvantage of thin client is _____
4. The main data synchronization formats are _____

13.4 DATA SYNCHRONIZATION AT CLIENTS AND SERVERS

The mobile device gets the updated data such as messages, emails, app data from the server. The data is synchronized between the server and the mobile client securely and regularly.

In this section we discuss various topics related to synchronization such as synchronization types, usage models and others.

13.4.1 Synchronization Types

The mobile client and the remote server use various synchronization methods to update the data. Let us look at the different types of synchronization –

Two-way synchronization

In a two-way synchronization, the data is synchronized bi-directionally between mobile client and the server. Whenever the mobile client connects to the server the updated data is synchronized with the server.

For instance, when the user adds a new contact in the mobile device, the updated contact details are synchronized with the remote server when the mobile device connects to the server. Similarly if the remote server has an updated patch file, it synchronizes with the mobile device.

Synchronization on server alert

In this scenario, the data is synchronized from the remote server to the mobile client. Whenever the server gets a new data that is required for the mobile client, it alerts the mobile device. The mobile device then pulls the updated data from the remote server.

For instance, when a new message arrives for the mobile device at the server, the server alerts the mobile client which then pulls the updated message from the server.

Server initiated synchronization

The server initiates the synchronization when the data is updated on the server end. The updated data is identified based on its last modified timestamp. During the synchronization, the updated data is synchronized to the mobile client.

For instance, the server initiates the synchronization when a new email arrives for the mobile device. Upon receiving the email message, the server pushes the email to the mobile client.

Synchronization upon client request

The mobile client requests the server to refresh its data. For instance, the mobile client requests the server to refresh its locally stored spam list or any of the mobile app configuration file. Upon this request, the remote server synchronizes the latest data for the spam list and pushes the latest configuration file.

Client initiated synchronization

In this scenario, the client pushes the updated data to the remote server. For instance, the application configuration, user preference data, analytics data, click-stream data is pushed from mobile client to server when the data is updated.

A variant of this model is that client can back up the local data on the remote server. Whenever the data is updated, the mobile device backs up the data remotely.

Generally during the synchronization, only the updated data is synchronized from the source to the target. In a bi-directional synchronization, the synchronization system compares the data on both client and the server and uses conflict resolution rules to update the data. If the data is updated at both client and the server, the data is either merged or overwritten based on the preference.

13.4.2 Synchronization Usage Models

A typical mobile computing system consists of various components such as the following:

- Mobile client that interacts with the mobile computing ecosystem by invoking APIs
- Mobile carrier that provides necessary network coverage for data transmission.
- A local PC or remote sever that connects to the mobile client for data backup.
- A resource such as Bluetooth speaker or a smart TV to which the mobile client connects.

The mobile client employs multiple usage models to synchronize the data with the mobile computing ecosystem. We have detailed the main usage models in this section.

API based synchronization

In this method of synchronization, the mobile client API synchronizes the data with a remote server or a resource through API running on the target device. We have depicted the API based synchronization in Figure 4.

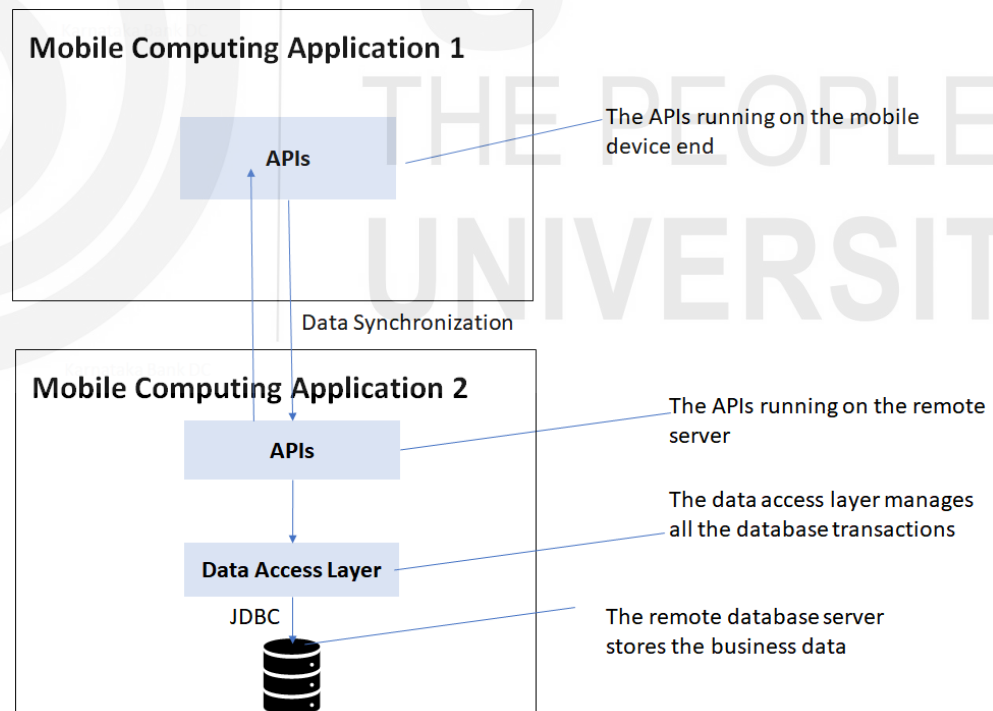


Figure 4 API based synchronization

For instance, a user profile management API running on the mobile device synchronizes the user profile updates to the remote server.

Synchronization to nearby PC

In this method the mobile device synchronizes the data with the nearby device such as personal computer (PC) or a Bluetooth device. We have depicted the synchronization in Figure 5.

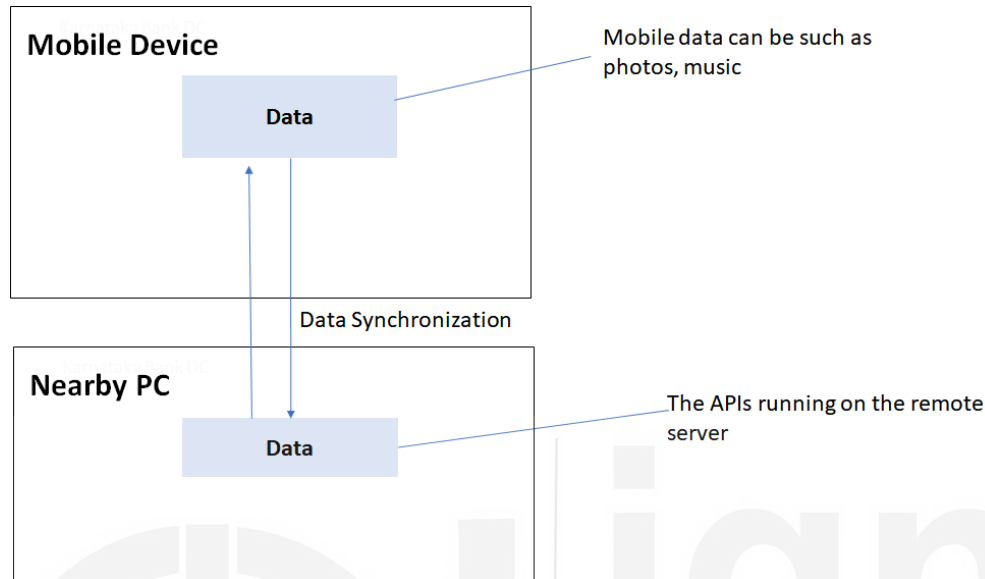


Figure 5 Synchronization between mobile device and nearby PC

For instance, the mobile device client synchronizes the files or music videos to the nearby PC through USB or Bluetooth.

Synchronization to remote server

In this synchronization method, the mobile device connects to the remote server through WIFI or wired Internet and synchronizes the data. We have depicted the remote server synchronization in Figure 6. A variant of this approach is that mobile device uses an intermediate system (such as local PC) to synchronize the data with the remote server.

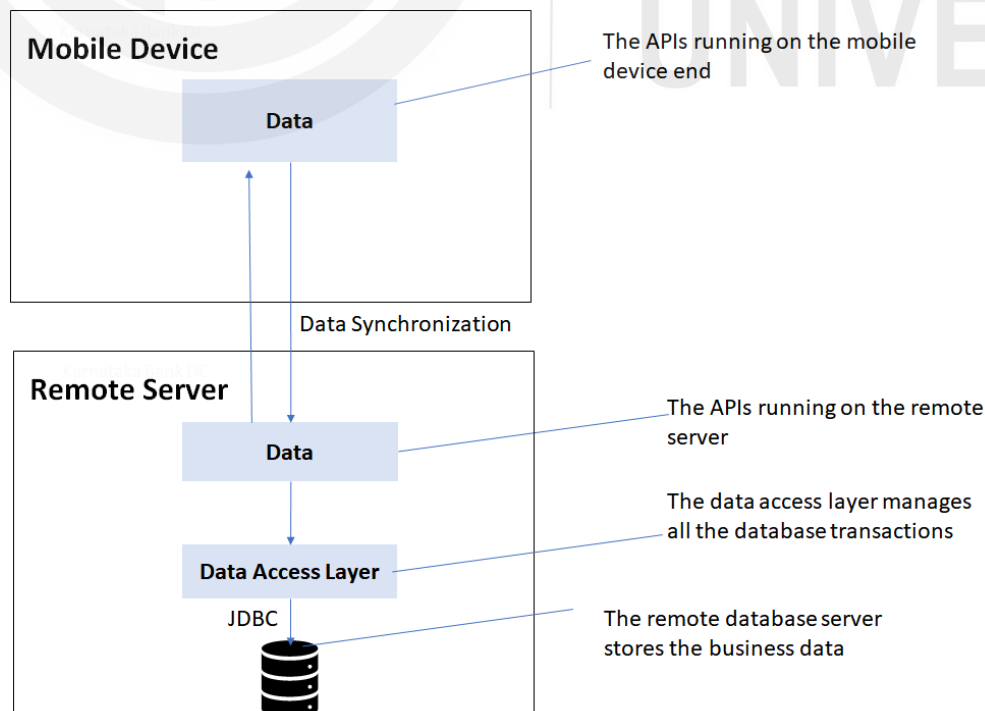


Figure 6 Synchronization with remote server

For instance, the mobile device backups the messages to the remote cloud server.

13.5 MOBILE DEVICES SUPPORT INFRASTRUCTURE AND MANAGEMENT

An enterprise needs a centralized device management platform to manage the heterogenous mobile devices. Given below are the main features of a device management platform –

- Bootstrapping the device with initial configuration
- Monitoring the configuration changes
- Handling the device maintenance
- Managing the device location and device handover

In this section we discuss the mobile device support infrastructure

13.5.1 Mobile Device Support Infrastructure

The mobile service provider should manage the wireless infrastructure (such as network coverage, mobile SIM registration, call management and others). If the mobile apps need additional security features such as multi-factor authentication (MFA), then the device support infrastructure should manage those requirements as well.

The mobile device also should work transparently from the device management team. When the mobile device starts up, it should register itself to the network. The mobile device should perform activities such as setting up the communication, initiating the call, terminating the call, closing the communication and such. We have depicted the mobile support infrastructure in Figure 7.

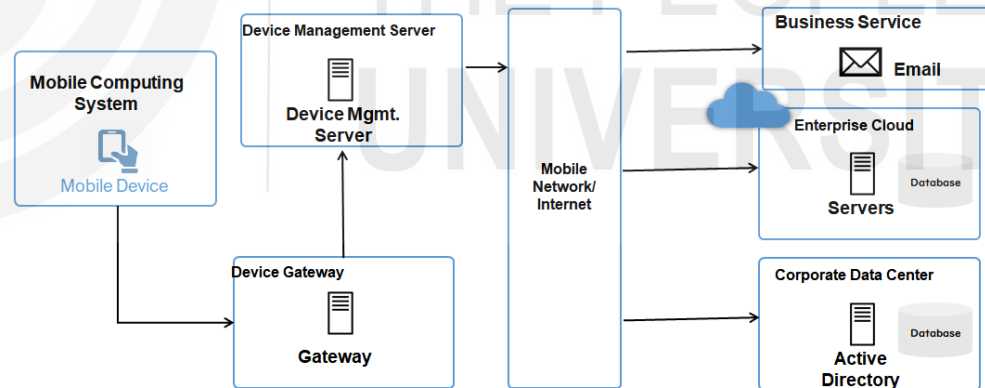


Figure 7 Mobile Device Support Infrastructure

The main steps of the mobile support infrastructure is as follows –

1. The mobile computing system (such as mobile devices) connect to the device gateway. The device gateway acts as an intermediary between the mobile client and the device management server.
2. The device management server assigns a unique id to all the registered mobile devices. The mobile device details are tracked using the unique id. If the mobile device moves to a different location, the id is changed accordingly.

3. The device management server connects to the Internet.
4. Through the Internet the mobile device can connect to enterprise data center (which is hosting the enterprise database and services), enterprise cloud platform (which is hosting the cloud services and servers) and business services (such as email, ERP and such).

☛ Check Your Progress 2

1. Bi-directional synchronization happens in _____
2. The main synchronization usage models are _____
3. Server pushing the new email to mobile client is an example of _____
4. Mobile client requesting to update its configuration file is an example of _____
5. _____ acts as an intermediary between the mobile client and the device management server.

13.6 SUMMARY

In this chapter we discussed various kinds of mobile architecture. There are mainly three types of mobile related architectures- Thick client architecture, thin client architecture and smart client architecture. The thick client architecture encompasses all the tier components in a single structure. Thick client does not do server calls. Thin client architecture involves minimal components on the presentation tier and invokes backend services for the data. Smart client applications are thin clients that use local database storage for managing the data. Smart clients provide rich user interface, effective user experience and leverages native mobile features. The main data synchronization formats are Open Standard Formats, Flat File format, Database format and Device specific storage. The main synchronization types are Two-way synchronization (wherein bi-directional synchronization happens between mobile client and the server), Synchronization on server alert (wherein the server alerts client for synchronization), Server initiated synchronization (wherein server initiates the synchronization when the data is updated on the server end), Synchronization upon client request (wherein the client requests the server to refresh its data) and Client initiated synchronization (wherein the client synchronizes the data to the server). The main synchronization usage models are API based synchronization, Synchronization to nearby PC and Synchronization to remote server. The mobile support infrastructure mainly consist of the device gateway, device management server and the Internet that connects to the enterprise servers.

13.7 SOLUTIONS/ANSWERS

Check Your Progress 1

1. Thick client.
2. Local storage
3. frequent server calls
4. Open standards, device specific storage, flat file format and database format.

Check Your Progress 2

1. Two-way synchronization.
2. API based synchronization, Synchronization to nearby PC, Synchronization to remote server,

3. Server initiated synchronization
4. Synchronization upon client request
5. Device gateway

13.8 FURTHER READINGS

References

https://en.wikipedia.org/wiki/Network_virtualization

https://en.wikipedia.org/wiki/Cloud_computing

